

Wander Whisper

Dung Pham, Derrick Nguyen, Khoa Tran, Sanket Makkar

Contents

I. Abstract.....	2
II. Background.....	2
III. Proposed solutions.....	3
IV. Success Criteria.....	4
V. Technical requirements and specifications:.....	5
A. Data Processing.....	5
B. Data Collection.....	5
C. Frontend.....	6
D. Backend.....	6
VI. Related work.....	7
A. Large Language Models:.....	7
B. Location Recommender Systems:.....	7
VII. Management.....	8
VIII. Methodology.....	9
B. AI Engineer (Derrick Nguyen).....	11
C. Data Scientist (Dung Pham).....	11
D. Backend Developer (Sanket Makkar).....	12
E. Frontend Developer (Khoa Tran).....	13
IX. Results & Analysis.....	15
X. Future Work.....	17
References.....	19

I. Abstract

Traditional travel recommendation systems require users to select a destination before offering suggestions, which limits spontaneous and exploratory trip planning. Sites like TripAdvisor only generate recommendations after a location is provided and lack the ability to handle diverse inputs like natural language or images for personalization. Wander Whisper leverages advanced machine learning models—CLIP for visual understanding and a fine-tuned T5 model for natural language processing—to combine multimodal inputs into tailored travel recommendations. It ranks destinations based on user interests and provides detailed suggestions, including flight options, local transportation, weather forecasts, and curated activities. The system also explains its recommendations to encourage user trust and engagement.

II. Background

Conventional travel recommendation systems, like TripAdvisor, mostly work by making activity and destination recommendations after a customer has already chosen a place. To make suggestions, these platforms employ historical behavioral data and organized user preferences. They are unable to assist consumers who are unsure about their destination choices in having an exploratory travel discovery experience, nevertheless. There is a gap in the current picture of travel suggestion technology caused by the lack of user-friendly and interactive systems that can help travelers in the early phases of trip preparation.

Furthermore, it is difficult for current recommendation systems to combine various input formats - such as text, graphics, and user-defined preferences - into a single suggestion process. The capacity of traditional methods, such as deep neural networks and collaborative filtering, to handle and comprehend intricate, multimodal user inputs is constrained. Consequently, travel suggestions continue to be limited and frequently fall short of encompassing the entire range of user interests.

Large Language Models (LLMs) have made it possible for algorithms to directly extract specific trip preferences from user-generated language, opening up new possibilities for improving travel recommendations. LLMs can evaluate natural language inputs and extract complex travel preferences, in contrast to previous approaches that frequently rely on standardized query-based interactions. The majority of LLM implementations in travel

recommendation systems today, however, merely function as basic query wrappers and do not fully utilize their potential for suggestion optimization and database retrieval.

The lack of interpretability and transparency in recommendation outputs is another serious flaw in current platforms. Travel recommendations are frequently given to users without providing explicit justifications for the selection of a specific location. Because travelers could not fully comprehend how a system derives its recommendations, this lack of interpretability might result in lowered user trust and engagement. Improving user experience and building confidence in AI-powered travel discovery systems need increasing the transparency of the recommendation process.

A unique strategy combining multimodal input processing, explainability, and sophisticated AI-driven search capabilities is needed to address these issues. A next-generation travel discovery system can greatly improve how users explore and plan their trips by utilizing LLMs for natural language comprehension, incorporating image-based travel preferences, and offering concise explanations for recommendations.

III. Proposed solutions

In order to solve this, we suggest a multi-modal framework - Wander Whisper, allowing users to input their travel preferences and images representing ideal destinations characteristics. Our recommendation system will then return top 10 user-personalized cities, including details such as flight tickets comparison, transit locations, weather, flight duration, recommended activities and explanations on why this destination fits that particular user needs. Our strategy incorporates explainability elements, which give users context for a particular recommendation and promote usability and trust.

Furthermore, by integrating geographical accuracy, trip distance, affordable flight tickets and time calculations, and an extremely user-friendly interface that outperforms current solutions like TripAdvisor and Airbnb but has a genuinely global reach, an interactive user interface would improve the travel planning process. The application's workflow can be seen from *figure 1*. Our code will be publicly available at these github repositories for the [backend](#) and the [frontend](#).

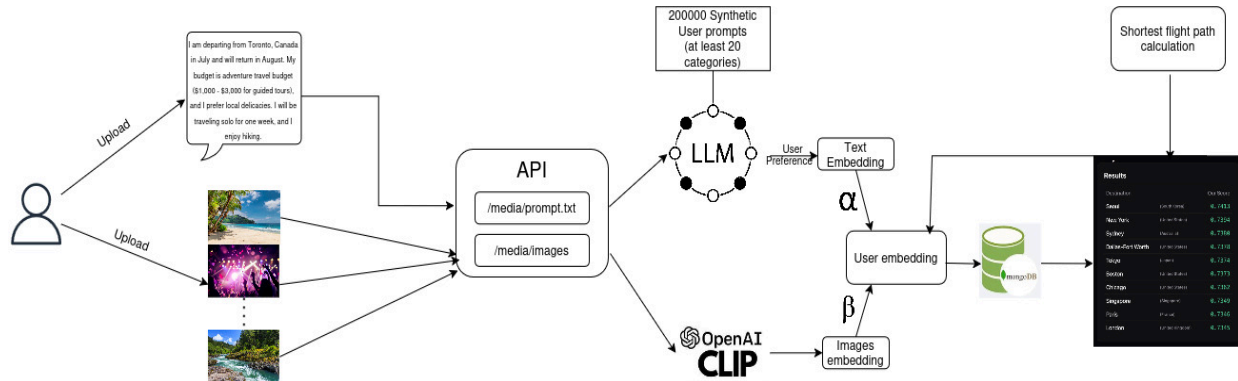


Figure 1: Wander Whisper Framework

IV. Success Criteria

To ensure that we actually meet the goals of this project, we create a set of requirements that must be met.

Highest of all is that total inference time along the entire backend processing pipeline — from first user input through to final recommendation output — is less than or equal to 10 seconds. This performance requirement is essential to providing an interactive and seamless user experience, especially as the system increases to handle lots of concurrent requests.

Second, the MongoDB database must be populated with an appropriately varied and representative sample of city data. Specifically, the database must be spread over at least three continents, and for any continents that are selected, there must be data on at least 2% of available cities using legitimate datasets. By doing so, our recommendations are rendered universally applicable rather than geographically directed towards some particular narrow subset.

Apart from text inputs, the system would also be able to upload any image in PNG and JPG format through the front-end. The functionality enables the system to handle a wider context of information, hence richer and more diverse user interaction.

For every tuning process on synthetic prompt generation, no fewer than 10,000 synthetic prompts must be created. The task is to amplify the training content sufficiently such that responses under actual use by the model become improved, believable, and transferable.

Finally, the user interface must present explainable and understandable outputs. All the city suggestions given to the user must be accompanied by an equivalent score of appropriateness or relevance. The system must also contain a text explanation for each

suggestion that informs the user of the rationale behind the choice on the basis of user information and other considerations. This is done so that the trust of the user is obtained and users are allowed to make very informed decisions as per the recommendations given by the system.

V. Technical requirements and specifications:

A. Data Processing

Contrastive Language-Image Pretraining (CLIP) is a multimodal model that will learn visual information from natural language supervision developed by OpenAI. For this project, we aim to use CLIP to extract the features from images that users upload to merge those information with the textual information from the input prompts, then integrate those insights into our recommendation pipeline. Users will be able to upload multiple images that represent their travelling preferences. Those images will then be resized and normalized before passed into the CLIP model.

T5 is a language model that we would use as an information extraction module. We fine-tune the model with synthetic data of user input examples to improve its ability to extract information from textual prompts. The model will then process the text to find key information regarding their travel preferences such as budget, activities, weather preferences and so on. These information will be merged with the image features generated from CLIP and integrate into our recommendation framework to generate personalized travel recommendations.

B. Data Collection

A major challenge of this project is finding or creating a comprehensive dataset for matching and recommending travel destinations. Since we want to tailor our recommendation to all of our users' preferences and needs, we require a rich dataset that contains attributes such as City Name, Country, Latitude, Longitude, Tourist Attractions, Tourist Attraction Reviews, Weather, Safety Index, Cost of Living, Cuisine & Restaurants, Language Spoken, Visa Requirements, Best Travel Months, and City's Characteristics, etc.

We fine-tune our generative model using a synthetic dataset. This dataset consists of synthetic user travel preferences in the form of structured prompts. These prompts will help the model to extract key attributes from a set of extraction tasks.

C. Frontend

We chose React.js for the fundamental technology, specifically employing typescript for the front end system's flavor, mostly because of the framework's versatility. We will be able to build a scalable and consistent front-end system with the components feature. We will use Tailwind CSS for the app's styling, mostly for its modularity and consistency. Tailwind offers low-level utility classes that may be created directly in our HTML class, in contrast to conventional CSS frameworks. Tailwind allows us to spend less time on CSS and more time on other application development tasks. We are selecting Vercel for deployment and hosting because of its convenience. Automatic deployments straight from our Git repository and the smooth integration with React are also key factors in our decision. With the ease of deployment, our team can concentrate on polishing the software, which guarantees dependable, quick, and easy deployments to production.

D. Backend

Django is a web framework written in python that is incredibly popular. For this project, Django will be used largely in providing API's to expose underlying functionality of the project to the front end. In particular, exposing the underlying extraction logic, fetching an array of external data from the web, and path planning will be the primary use-cases for the Django framework in this project. Django essentially is what will allow this project to tie together, and will be what is used to host all backend work for this project. Heroku is a cloud platform (similar to AWS for applications) which allows us to host applications on external and fast servers. Heroku is what will be used to host the django application created, and will essentially be where non-front-end efforts live.

Use of docker for a consistent safe virtual environment may be warranted in future iterations of this project - though for the scale of an individual project that is only to be hosted on the one server this may be overkill. We will keep this option in consideration for future scalability.

Among internal API's, one API to request user input be translated into top city mappings will be exposed. Additionally, a route planning API, and additional utility API's will also be exposed. For external API's, we will expose one among various travel pricing and planning API's through a custom formatted api in the Django app, as well as API's that grab weather,

humidity, and city review information. Current candidates include Amadeus Travel’s family of API’s, the OpenWeatherMap API, and perhaps the Yelp API set for reviews.

VI. Related work

A. Large Language Models:

Nowadays, everywhere you look and every news headline you read or watch, AI seems to be making an appearance. One such type of artificial intelligence with the capability of understanding, processing, and generating human languages are Large Language Models. Also known as LLMs, these large language models are advanced deep learning systems trained on massive datasets. Underneath the many complex layers, the basic building blocks of these systems are sets of neural networks that, in short, extract meaning and context from sequences of texts (Malervy, 2024). These LLMs are crucial for the age of generative AI, powering models like ChatGPT, Gemini, Claude, etc. While all of these are impressive, we have chosen to work with T5 and fine tune it to better suit our needs. The aforementioned LLMs are all pre-trained on massive data sets. While this is very impressive and useful, we believe that for a smaller use case like ours, fine tuning T5 will be more appropriate. As described above, our approach involves converting text inputs into embeddings. We will then categorize them into specific attributes to feed them into our developed recommendation system to match with our existing data set.

B. Location Recommender Systems:

The vast global flight network allows anyone to go anywhere in the matter of days or even hours. Most existing frameworks focus on intra-city recommendation rather than suggesting a personalized destination. Traditional recommendation systems primarily focus on collaborative filtering and content-based filtering to find activities, hotels, and restaurants in a particular city (TripAdvisor, Yelp, Google Travel). However, these methods assume the traveler already has a destination in mind, overlooking the process of selecting a city that fits the user preferences most. Recent research has also looked at multimodal approaches for route optimization. SelfAM-Vtrans (Frontiers, 2024) and Dynamo (Subramaniaswamy, 2018) integrate visual and textual data to optimize travel routes within a city and suggest personalized paths. These methods focus on the transportation logistics and activity sequencing of a city, neglecting the

general information that users look for in a destination such as weather, budget range, crime rate, etc. By merging textual input, visual cues, large language model, and external metadata for practical constraints, our system not only ranks popular locations based on reviews but also tailor to each individual, filling a key gap in existing travel recommendation frameworks. Our model's personalization distinguishes it from existing systems, enabling a more comprehensive travel planning experience for users from anywhere.

VII. Management

We divided our group into 2 small teams: the data team and the developer team. In our data team, we divided it into 2 smaller roles: data scientist (Derrick Nguyen) and data engineer/database administrator (Dung Pham). In our developer team, we divided it into 2 smaller roles: front-end developer (Khoa Tran) and back-end developer (Sanket Makkar).

On the managerial side, we plan to schedule weekly meetings on Wednesdays from 10:00 to 11:30 AM, with deliverables aligned to a two-week sprint cycle, as below:

Time	Deliverables & Due Dates
Jan 29th	- Finalized Project Ideas - Individual Research, Database + API
Feb 12th	- Work on user embedding, recommendations and explanations - Path planning algorithms and city-airport data extraction finalized - Finish sketching, planning and architecting - Dataset generation
Feb 16th	- API path planning integration - Data forwarding API work for basic text finalized
Feb 16th	- Data forwarding for image, alpha-beta values finalized - Code refactoring for simplicity

	- Media directory orchestration finalized
Feb 26th	- Finalized Backend and Database connection
Mar 5th	- Finished Developed Frontend without API call to the backend - Output generation for top-K results - Prepare for Demo
Mar 12th	- Complete linking the frontend with the backend
Mar 16th	- Ensure our code base works across all platforms - Starting on testing and documentations. - Finalize Dataset with desired attributes - Test inference with different CLIP model and fine-tune T5 parameters
Mar 20th	- Finished testing and documentation
Mar 26th	- Implement User's login and profile system - Prepare for Intersections
Apr 9th	- Finish implementation of user feedback loop - Implement User's login and profile system - World tour feature - Implement sequential location recommendation for user based on recorded history
Apr 23rd	- Finalize Presentation and final Report

Table 1: Deliverables & Deadlines

VIII. Methodology

A. Team

To implement the backend of this project, we broke down our work into standard backend API components, AI components, and database components to produce one comprehensive recommendation pipeline as shown below in figure 2.

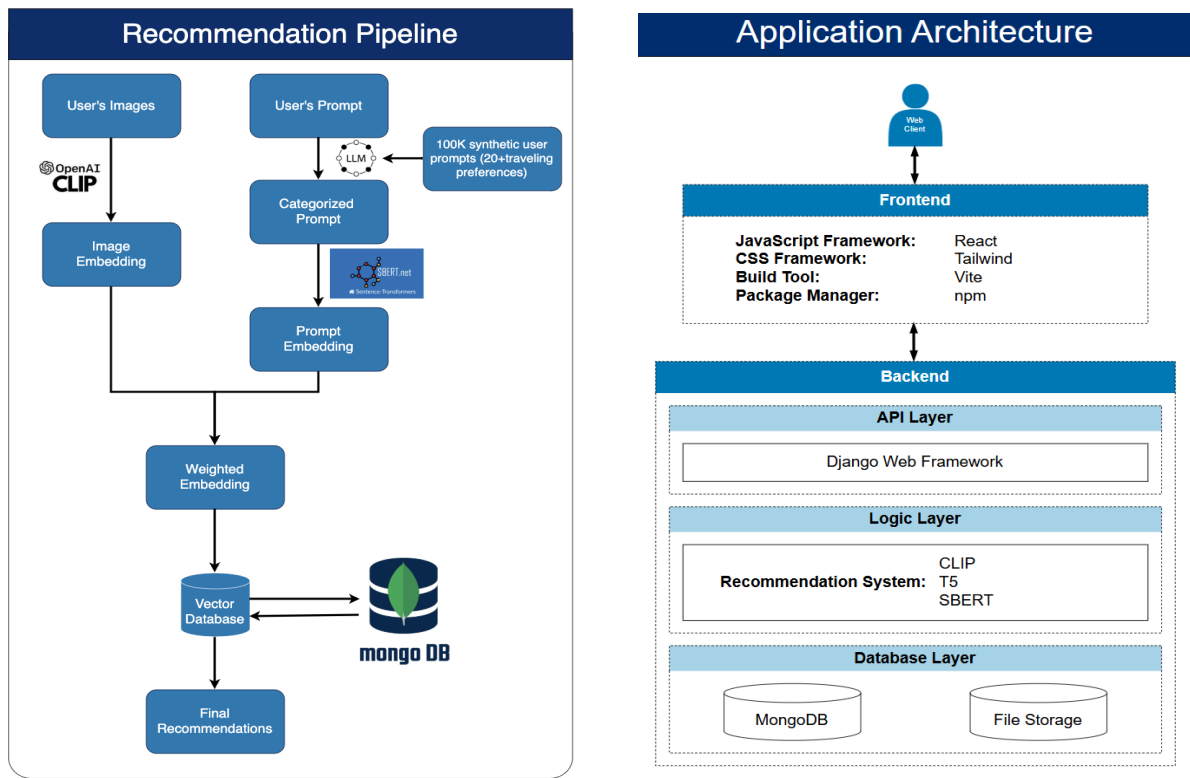


Figure 2:
Recommendation Pipeline & Application Architecture

The flow of information then is as follows. The user, via the front end, makes API calls to upload images, stored in an external media directory, and or upload their text prompt, also stored in a separate media directory.

When a request for recommendations is made, user images are run through CLIP for image embeddings. Simultaneously, a custom fine tuned T5 model will take a user prompt and generate a json file from this prompt before sending off this json file to SBERT, a model that generates a text embedding. The weighted sum of text and image embeddings, set by the user optionally changing the parameter before the request, is then computed.

This weighted sum is then run against a vector database of city data contained in our managed MongoDB database for a final city recommendation to be made. Along with city names city similarity score, city names, latitude and longitude, country data are returned. To add to the

quality of the response, the backend additionally computes shortest travel distance between cities by airports and supplies this auxiliary information to the front end.

Our responsibilities for implementation of this project are then as described in the following sections of project methodology.

B. AI Engineer (Derrick Nguyen)

My main responsibility has been to process user input and uploaded photos in order to produce customized travelling suggestions. I implemented a pipeline that uses a fine-tuned T5 Language Model to scan user-submitted text prompts and extract structured travel preferences. These are then converted into text embeddings that reflect the user's preference. Implemented concurrently, OpenAI CLIP is used to extract picture embeddings from uploaded images, so collecting visual travel preferences that might not be stated directly in language. Then, I developed a top-k city ranking system that takes user preferences and constraints into account while ranking possible travel destinations. In order to offer insightful answers, customized explanations for every suggestion were also created. A final user embedding is then produced by combining these embeddings using the weighting parameters α (image) and β (text). I then created an ordered list of suggested locations by using Facebook's FAISS library to quickly obtain the most comparable city embeddings based on user input. Then, a design of a proposed tour itinerary based on the user's preferences and travel restrictions and the destination they have chosen was made.

C. Data Scientist (Dung Pham)

To start, I've been able to find a few datasets related to cities, tourisms, and characteristics. From these big data sets with thousands of entries, they were narrowed down into two smaller datasets: US_cities and top_500_cities. After data handling, scraping, generation, filtering, and merging, these two data subsets have the following tables and attributes, as seen in figure 3: city_id, name, description, weather, landscape, activities, cuisine, purchase_power, health_care, pollution, quality_of_life, crime_rating, country, region, lat, lng, population, and img_folder. My next step was to integrate MongoDB into our script. I created a cluster on MongoDB atlas and uploaded our data. My goal for MongoDB is for efficient data storage, querying, and scalability. A script was then written to pull the data from the Database for

embeddings and index generation with FAISS. The resulting city.index file enables fast similarity searches. Another script was then developed to extract the top k most relevant cities based on similarity search of the user's input embeddings. Lastly, our recommend_cities script connects to our cloud database to extract the corresponding attributes and compiles into a .json file for front end's purposes.

Afterwards, my main focus is to enhance the data for practical use. The final dataset has a few thousand cities, each enriched with a more in depth and relevant list of attributes. Such attributes are city_type, attractions, and other tourism-related information. Along with the final dataset, I want to have a collection of smaller subsets for each continent or region for more targeted use. Another thing on my list was to optimize the existing scripts and improve error handling for various edge cases in both the data and scripts.

```
{
  "name": "Tokyo",
  "city_id": 1,
  "metadata": {
    "short_description": "A bustling metropolis known for its modern technology, culture, and vibrant city life.",
    "weather": {
      "winter": "cold",
      "spring": "mild",
      "summer": "hot and humid",
      "fall": "cool"
    },
    "landscape": "urban",
    "activities": [
      "temples",
      "shopping",
      "technology exhibits"
    ],
    "cuisine": [
      "sushi",
      "ramen"
    ],
    "purchase_power": "47.57",
    "health_care": "71.53",
    "pollution": "30.54",
    "quality_of_life": "69.29",
    "crime_rating": "13.91",
    "country": "Japan",
    "region": "ASIA",
    "lat": "35.6897",
    "lng": "139.6922",
    "population": "37732000"
  },
  "image_folder": "./images/tokyo"
},
```

Figure 3: Wander Whisper's Data Outline

D. Backend Developer (Sanket Makkar)

I used Django to develop clean backend code to efficiently store and handle user input including text upload, image upload, and parameter setting - all of which was accomplished via Media handling logic and a simple Django model to persist user settings.

Furthermore I worked to provide protection on the backend side from bad user input, advanced HTTP responses to the user to encourage correct usage of each API, consolidated database querying and embedding extraction logic, and managed to eliminate a number of path problems and errors that came from integrating backend and data science efforts.

Additionally, I made a significant effort to path planning logic - which was also exposed via API, by developing core logic to locate online databases that contain information regarding flights information between airports, extract and store this information, and efficient search algorithms to locate the shortest path from any one city to each other city via airport information.

Finally I assisted in integration efforts on the frontend with backend services. Concurrently with significant refactoring of the data-science components to this project for clarity, I pursued efforts to simplify the backend codebase for greater readability and pushed forward optimizations in terms of the number of API calls required.

E. Frontend Developer (Khoa Tran)

I will divide the frontend development process into four stages: planning, developing the user interface, and testing. Each stage will last roughly one to two weeks long.

First, the team's main goals throughout the planning stage will be to collect user requirements, specify user processes, and produce wireframes/prototypes. We started out by having meetings to identify functionalities and the overall look and feel of the product. I first drafted out the overall design of the webpage through Figma, this is a cumulative design process that took around 2 weeks to complete, where after each meeting, feedback from the team members are collected to refine the design. The design from the Figma can be viewed through [this link](#). From the design, I then spent a session drawing out the architecture of the website, i.e. the framework used, the structure of the website in terms of pages and a checklist of what to do.

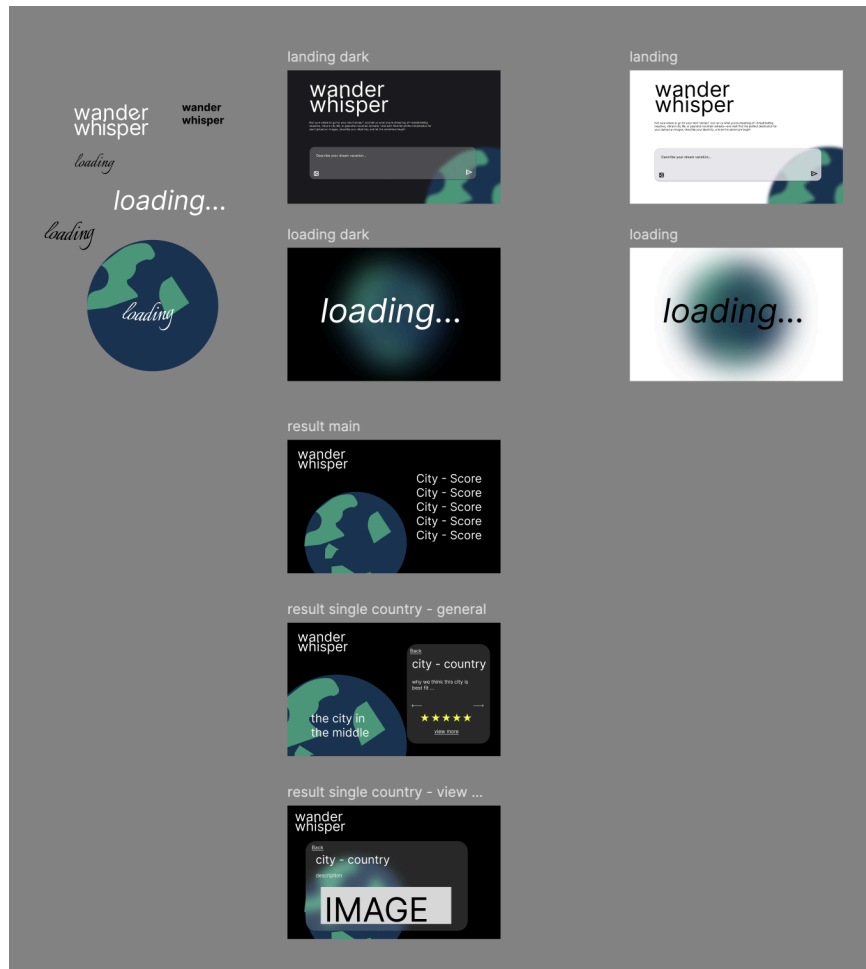


Figure 4: Figma Design

In the second phase, I constructed the application's main pages and components during the UI development stage, focusing on translating the Figma designs into a responsive, interactive interface. This includes creating reusable React components all styled using Tailwind CSS for a responsive and scalable design. React Context will be used to create state management in order to handle data, including user inputs and API responses. During this development phase, dummy APIs will be used to connect the frontend to the backend until all of the APIs are ready for use. Overall, I have been able to follow the initial design and planning very well.

Lastly, the testing stage guarantees that the application is accessible, functional, and free of bugs. I created unit tests for each component, including edge cases for state management and input validation. Integration tests have also been conducted to examine our actual backend APIs and confirm how components communicate with one another. To guarantee a consistent product

across platforms and devices, cross-browser/device testing has also been carried out, where I used an emulator to emulate devices with different sizes, making sure if all components are scale correctly.

IX. Results & Analysis

A. Recommendation System Evaluation

Component	Mean Time (s)	Std Dev (s)
Image Embedding	0.478	0.05
Text Embedding	2.996	0.302
User Embedding	3.126	0.359
Recommendation (FAISS)	0.421	0.042

The average running time of our recommendation system is 3.547. Most of the running time is spent on extracting the final user embedding representations.

Surprisingly, the running time of OpenAI CLIP is only around half a second for different CLIP variants with our use-case of uploading 2-5 images having negligible differences on the run time given the same variant.

Since we are fine-tuning the t5-base model, the running time to extract the information from the synthetic text prompt takes up most of the total time. This time also depends on the length of the prompt (depending on the number of categories in each prompt). Since the image and text embedding extraction are two separate tasks, we are able to run them in parallel and obtained the weighted final user embedding in 3.126s (average out of 100 runs)

FAISS is famous for its quick indexing search with the average similarity searching time being 0.421s in our 500+city database.

B. Application Snapshots

The application came out very close to what we originally planned, here are some snapshots of the UI. Figure 6 below shows the landing page of the application, the first page user will see when entering the application.



Figure 6: Landing Page of the Application

When the user needs to insert the picture, they can insert it through this panel as shown in figure 7.

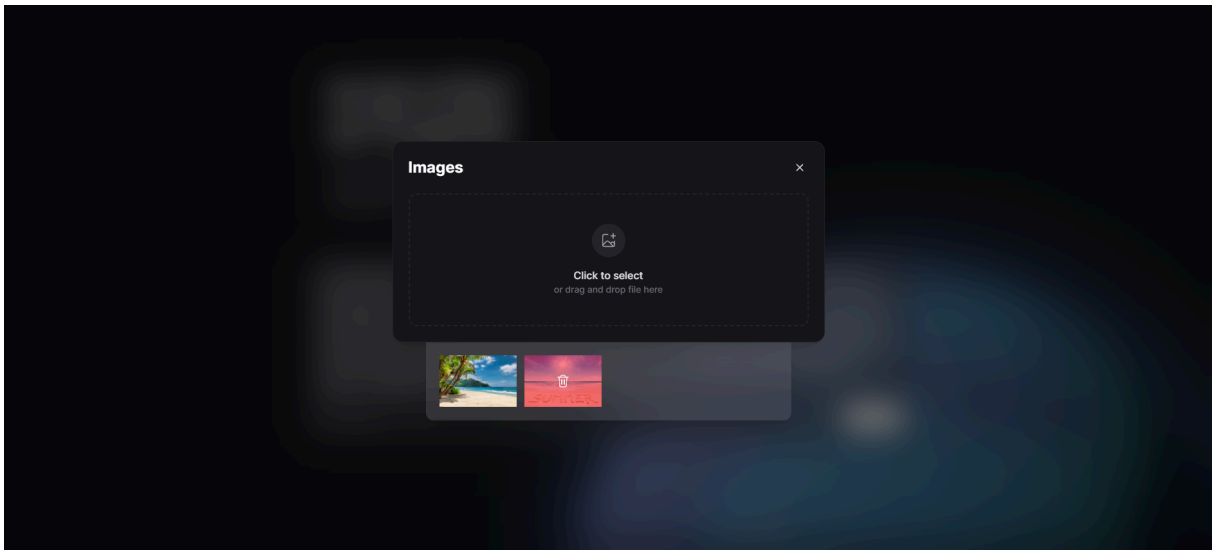


Figure 7: Image Insertion Section

After receiving all prompts and images necessary for a search, a list of cities will be rendered on the globe with their corresponding score. The globe is 3D rendered and fully intractable, allowing users to spin, zoom-in and zoom out (as shown in Figure 8). When the user decides to click on the city to learn more about the destination, the globe will zoom in at the specified city with extra description about the city, it can be seen in Figure 9.



Figure 8: Main Result Page

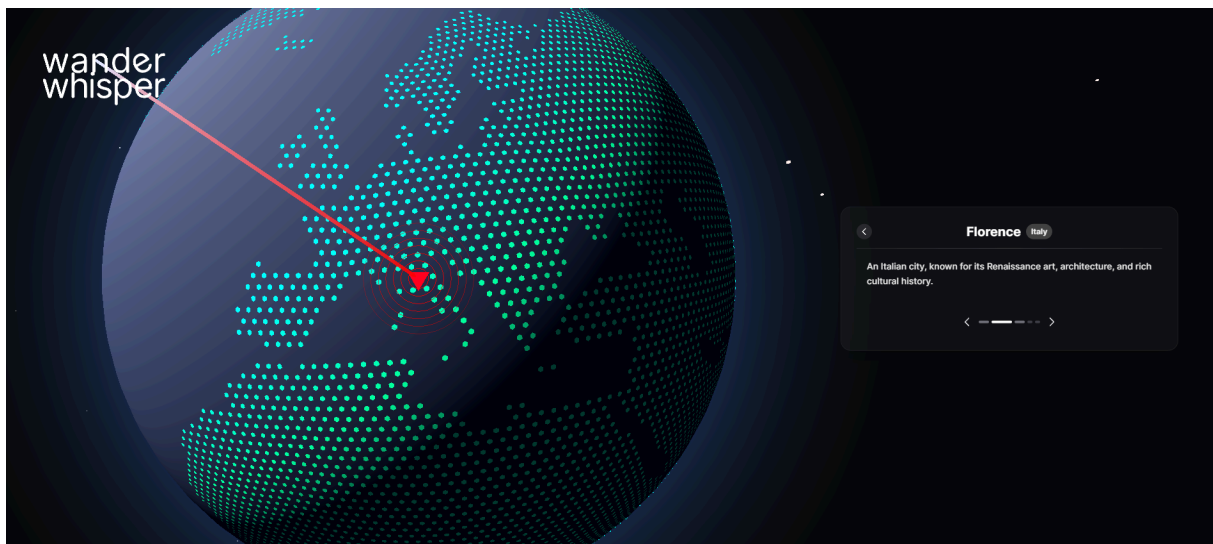


Figure 9: Single City Description

X. Future Work

Furthermore, we have planned several key possible add-ons to enhance functionality, personalization, and user engagement if time allows. First, we plan to incorporate social media feeds and reviews, enriching the platform with community-driven insights and fostering trust through peer validation. With this feature, we would have to incorporate scalability to our application that can handle an increasing number of users. Another feature will possibly be a "next location" suggestion system, which analyzes sequential travel history and preferences to

propose logical, personalized itineraries - such as recommending a coastal retreat after a mountain trek or a cultural city following a nature-focused trip. A final feature we plan to implement is the “world city tour” where an optimal path planning algorithm between all cities (city A to city B, city B to city C, and so on...) is shown on the globe through a simple toggle - giving the user the power to see what a world tour of all the cities they want to go to would look like. Lastly, we hope to push our frontend and backend API's to Vercel for hosting, as this would finalize our product and bring the whole project together. Together, these extensions will transform our platform into a holistic travel ecosystem, blending practical tools, dynamic content, and intelligent LLM-powered personalization to create a cohesive, adaptive, and socially connected user experience.

References

- Juan, Z., Zhang, J., & Gao, M. (2024). A multimodal travel route recommendation system leveraging visual Transformers and self-attention mechanisms. *Frontiers in neurorobotics*, 18, 1439195. <https://doi.org/10.3389/fnbot.2024.1439195>
- Malervy, J. (2024, December 9). *From ChatGPT to everyday AI: What's an LLM and why should you care? (FAQs)*. AI GPT Journal. <https://aigptjournal.com/ai-resources/faqs/llms-power-ai-faqs/>
- V. Subramaniaswamy, R. Logesh, V. Vijayakumar, K. Keerthana, K. Rakini and B. Swarnamalya, "Dynamo: Dynamic Multimodal Route and Travel Recommendation System," 2018 International Conference on Recent Trends in Advance Computing (ICRTAC), Chennai, India, 2018, pp. 47-52, doi: 10.1109/ICRTAC.2018.8679443.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning Transferable Visual Models From Natural Language Supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*. arXiv:2103.00020
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140), 1–67. [arXiv:1910.10683](https://arxiv.org/abs/1910.10683)